IntelliPaat

Module 1

Introduction to Python & Variables

Agenda

1. Introduction to Python.
2. Python Installation and Setup
3. Different ways of running Python Program
4. Fundamental syntax rules.
5. Introduction to primitive data types.
6. Variables
7. Handling user interaction

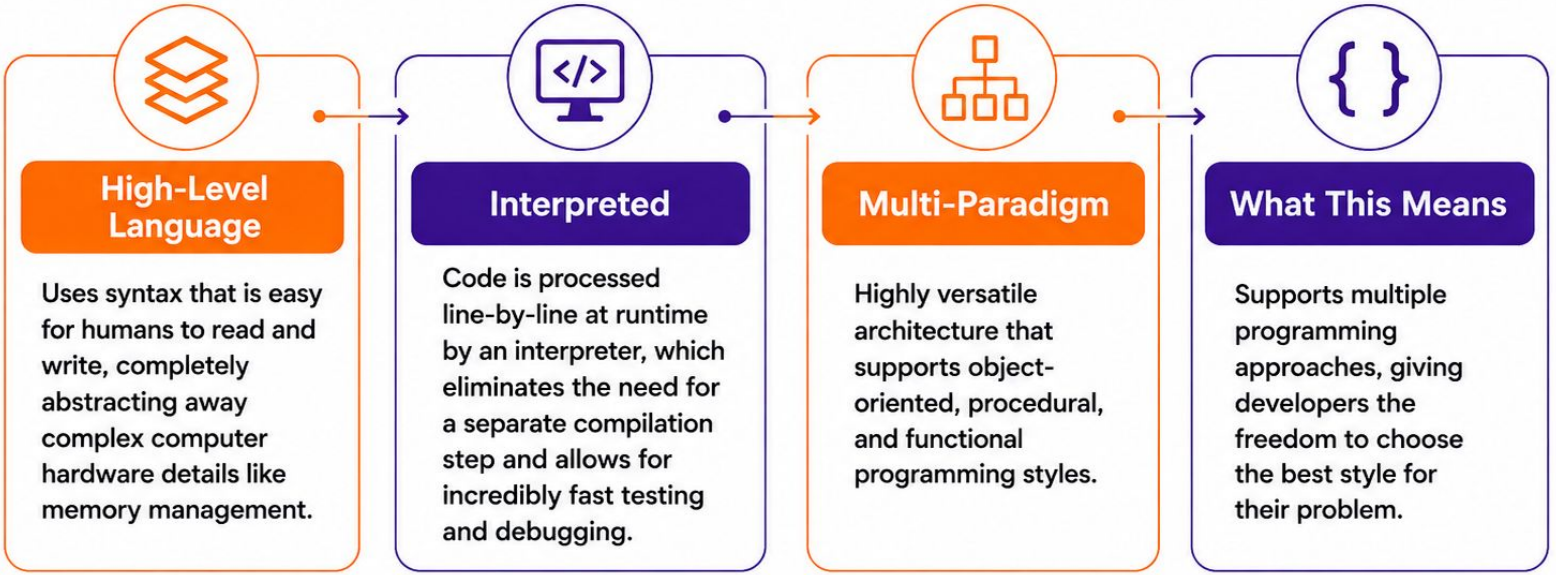
© Copyright Intellipaat. All rights reserved.

Introduction to Python

© Copyright Intellipaat. All rights reserved.

Introduction to Python

Definition: Python is a high-level, interpreted, general-purpose programming language celebrated for its code readability, simple English-like syntax, and extreme versatility.



© Copyright Intellipaat. All rights reserved.

Python Vs Other Languages

FEATURE	PYTHON	C	C++	JAVA
Level	High-level	Mid/Low-level	Mid/High-level	High-level
Syntax Complexity	Very simple	Moderate	Complex	Moderate
Performance	Medium	Very Fast	Very Fast	Fast
Typing	Dynamic	Static	Static	Static
Ease of Learning	Very Easy	Medium	Hard	Medium
Portability	High	Medium	Medium	Very High
Main Use Cases	AI, Data Science, Automation, Web	OS, Embedded Systems	Game Engines, High-performance Apps	Enterprise Apps, Android
Popular Frameworks / Tools	Django, TensorFlow	GCC	Unreal Engine	Spring Framework

© Copyright Intellipaat. All rights reserved.

How Python Executes Code

Definition: Python uses both compilation and interpretation.

Compilation

Compilation means converting the entire program into another form before running it.

Meaning	:	Converts the whole program into another form (bytecode) before execution.
Analogy	:	Translate the whole book from English to Hindi first, then your friend reads it.
When it happens	:	Before the program starts running.
Benefit	:	Faster execution once compiled.
Result	:	Bytecode (.pyc) is generated.

Interpretation

Interpretation means reading and executing the code line by line.

Meaning	:	Executes the program line by line at runtime.
Analogy	:	Your friend reads the book with you sentence by sentence.
When it happens	:	During the program execution.
Benefit	:	More flexible and easier to debug.
Result	:	Each line is executed one at a time.



© Copyright Intellipaat. All rights reserved.

Python Installation & Setup

© Copyright Intellipaat. All rights reserved.

Python Installation in Windows using IDLE

Follow the steps below to install Python and launch IDLE on Windows.

1. Go to python.org and download the latest Python version for Windows.

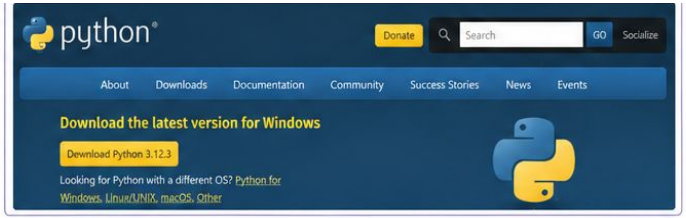
2. Run the installer and check the box **"Add Python to PATH".**

3. Click on **Install Now** and wait for the installation to complete.

4. Open IDLE (Python 3.x) from the Start menu.

5. The IDLE Shell window opens. You are now ready to write and run Python programs.

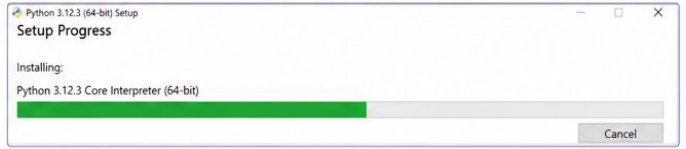
1



2



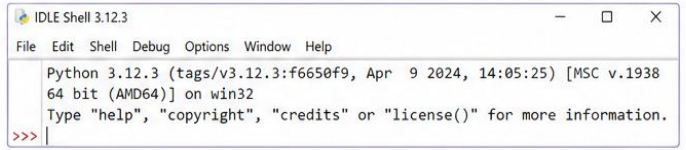
3



4



5



© Copyright Intellipaat. All rights reserved.

Python Installation using Anaconda + Jupyter

Follow the steps below to install Anaconda and launch Jupyter Notebook on Windows.

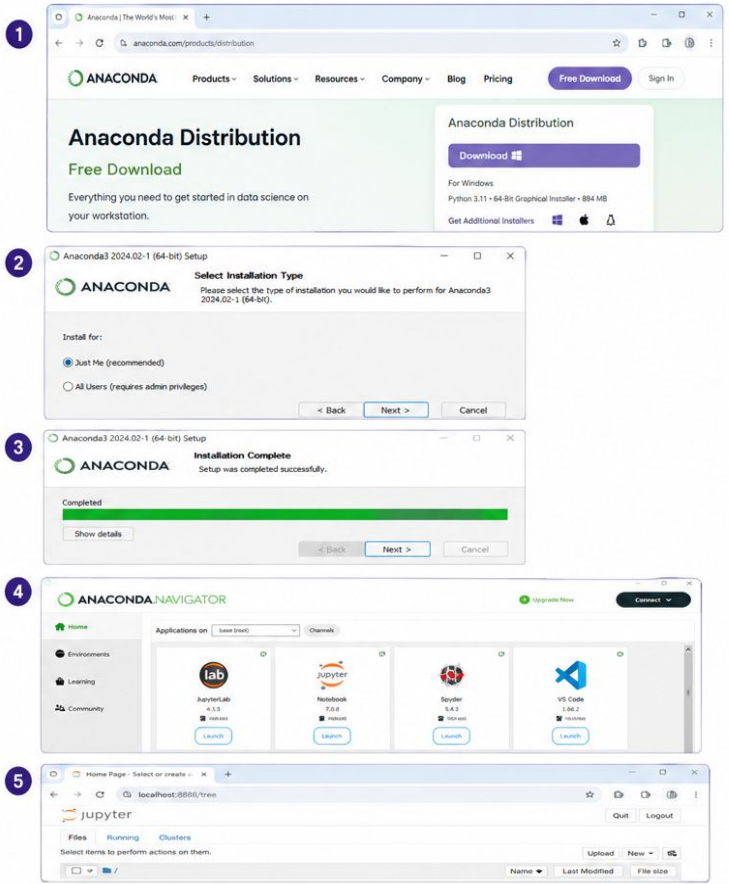
1. Go to https://www.anaconda.com/products/distribution and download the Anaconda Installer for Windows (64-bit).

2. Run the installer and click **Next**.
   Choose **"Just Me"** and click **Next**.

3. Select the installation location and click **Next**.
   Click **Install** and wait for the installation to complete.

4. Launch **Anaconda Navigator** from the Start menu.
   You will see the Anaconda Navigator dashboard.

5. Click on **Jupyter Notebook** in Anaconda Navigator to launch it. A new browser tab opens with the Jupyter interface. You are now ready to create and run Python notebooks.



© Copyright Intellipaat. All rights reserved.

First Python Program

Let's create your first Jupyter Notebook and write your first Python program. We will print "**Hello World**" on the screen.

1 Open Anaconda Navigator

- Launch Anaconda Navigator from your Start Menu.

2 Open Jupyter Notebook

- Click "**Launch**" under Jupyter Notebook.

3 Create a New Notebook

- Jupyter Notebook will open in your browser.
- Click on "**New**" → "**Python 3 (ipykernel)**".

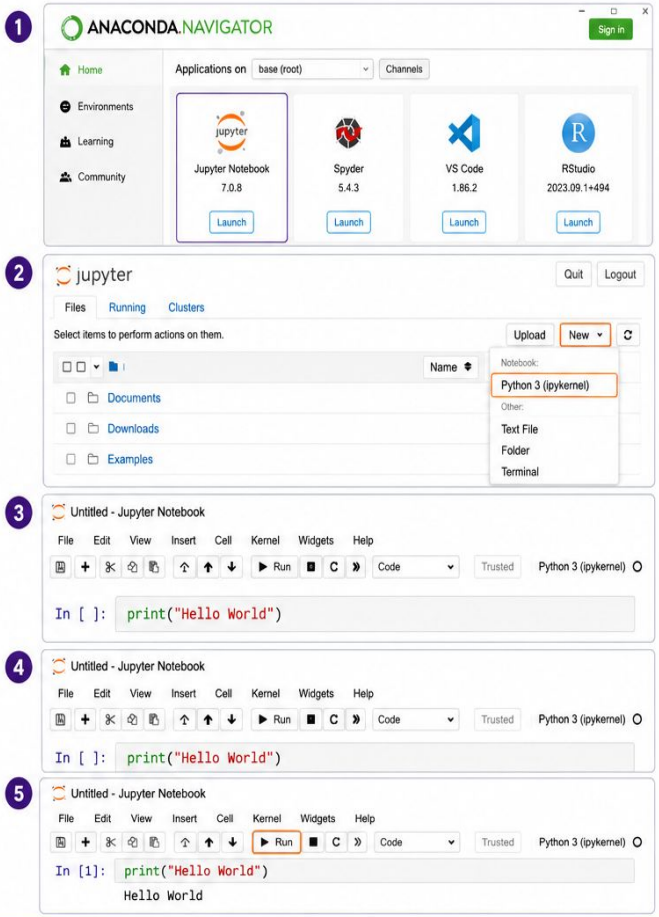
4 Write Your First Code

- In the first code cell, type the following code:

```
print("Hello World")
```

5 Run the Program

- Click the "Run" button (▶) or press Shift + Enter.
- You will see the output below the code cell.



© Copyright Intellipaat. All rights reserved.

Different ways of running a Python Program

© Copyright Intellipaat. All rights reserved.

Interactive REPL

REPL = Read → Evaluate → Print → Loop



It is an **interactive environment** where Python executes code immediately after you type it.

Example:

```
Python 3.11.4 (64-bit)

Python 3.11.4 (64-bit) on win32
Type "help", "copyright", "credits" or "license" for more information.

>>> 5 + 3
8
>>> print("Hello")
Hello
>>>
```

Best for:

- Quick calculations
- Testing small code snippets
- Learning Python basics

Advantages:

- Instant output
- Great for experiments
- Easy debugging

Limitation:

- Code is temporary (not saved automatically)

© Copyright Intellipaat. All rights reserved.

Saved .py Script

A script is a Python file saved with .py extension.

Best for:

- Large programs
- Projects and applications
- Reusable code

Advantages:

- Code is saved permanently
- Easy to organize
- Can run multiple times

Example file: program.py

```
1  name = "Adam"
2  print("Hello", name)
```

Bash

```
$ python program.py
```

Output

```
Hello Adam
```

© Copyright Intellipaat. All rights reserved.

Difference between Interactive REPL vs .py Script

Both ways are useful in Python. The table below shows their key differences.

Feature	Interactive REPL	.py Script
Execution	Line by line	Entire file
Output	Instant	After running script
Code Saving	✗ No	✓ Yes
Best For	Practice & Testing	Projects

Interactive REPL (Line by line)

Open Python REPL (Terminal or IDLE)

```
>>> 5 + 3
8
>>> name = "Adam"
>>> print("Hello", name)
Hello Adam
>>>
```

- Instant output
- Great for quick calculations
- Code is not saved automatically

VS

.py Script (Entire file)

1. Write code in a file (program.py)
2. Run the script (Terminal/Command Prompt)
3. Output after running script

```
1 name = "Adam"
2 print("Hello", name)
```

→

```
$ python program.py
```

→

```
Hello Adam
```

- Code is saved permanently
- Suitable for large programs and real projects

In Short: Use **Interactive REPL** for practice and quick testing.
Use **.py Script** for building real projects and saving your code.

© Copyright Intellipaat. All rights reserved.

Fundamental Syntax Rules

© Copyright Intellipaat. All rights reserved.

Indentation

Python uses **indentation** (spaces or tabs) to define blocks of code instead of { }.

- Indentation tells Python which statements belong together.
- Wrong indentation causes `IndentationError`.

CORRECT INDENTATION

Block of code under the `if` statement.

```
1  if True:
2      print("Hello")
3      print("Python")
4      print("Indentation is important!")
5  print("Done")
```

All statements inside the block are indented by the same number of spaces (recommended: **4 spaces**).

WRONG INDENTATION

No indentation after the `if` statement.

```
1  if True:
2  print("Hello")
3  print("Python")
4  print("Indentation is important!")
5  print("Done")
```

This will raise an error:
`IndentationError: expected an indented block`

KEY POINTS

- Use **4 spaces** per indentation level (recommended).
- Be **consistent** in your indentation throughout the code.
- Good indentation makes code **easy** to **read** and maintain.
- Wrong indentation leads to **errors** in Python.

Indentation is not optional in Python, it is **required!**

© Copyright Intellipaat. All rights reserved.

Comments

Comments are notes for humans;
Python ignores them during execution.

1 SINGLE-LINE COMMENT

Starts with #. It extends from # to the end of the line.

```
1  # This is a single-line comment
2  x = 10      # Assign 10 to x
3  y = x + 5   # Calculate y
4  print(y)    # Output the value of y
```

Everything after # is ignored by Python.

2 MULTI-LINE COMMENT (TRIPLE QUOTES)

Enclosed in triple quotes """ """.
Used for multiple lines of comments.

```
1  """
2  This is a
3  multi-line comment.
4  It can span multiple lines.
5  Used for documentation.
6  """
7  print("Hello, Python!")
```

USED FOR

Explaining code
Helps others (and you) understand the code.

Documentation
Used to write notes and descriptions.

Debugging
Helps in testing and finding issues.

GOOD PRACTICE

Use comments to write meaningful notes.
Keep comments short and clear.

AVOID

Don't over-comment obvious code.
Update comments when code changes.

NOTE

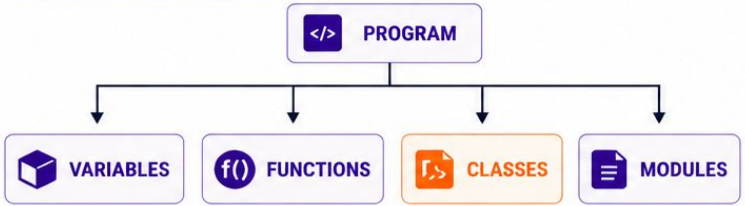
Comments do not affect the execution of the program. They are for humans, not for Python!

© Copyright Intellipaat. All rights reserved.

Code Structure

Python code is written top to bottom and organized using the following **building blocks**.

STRUCTURE HIERARCHY



ABOUT

- **Variables** store data values.
- **Functions** are reusable blocks of code to perform tasks.
- **Classes** are blueprints for creating objects (OOP concept).
- **Modules** are Python files (.py) containing code definitions that can be used in other programs.

1 VARIABLES

Purpose

Store data values.

Think of it as

A labeled container holding information.

Examples of data: name, age, marks, is_student, price, etc.

2 FUNCTIONS

Purpose

Perform a specific task.

Think of it as

A reusable action. It helps avoid repeating the same code.

Helps make code organized and easy to maintain.

3 CLASSES

Purpose

Create objects.

Think of it as

A blueprint or template used to design real-world entities like Student or Car.

It follows the concept of Object-Oriented Programming.

4 MODULES

Purpose

Organize code into files.

Think of it as

A toolbox of reusable code and definitions.

Helps keep large programs clean and manageable.

SUMMARY

Python programs are built using four main building blocks: **variables**, **functions**, **classes**, and **modules**. Understanding each block helps you write better, organized, and reusable code.



© Copyright Intellipaat. All rights reserved.

Case Sensitivity

Python is **case-sensitive**.
This means uppercase and lowercase letters are treated as different.

EXAMPLE

The following variables are **different** because of case:

```
1  name = "Manan"
2  Name = "Adam"
3  NAME = "Intellipaat"
4
5  print(name)
6  print(Name)
```

All three are different variables.

OUTPUT

```
1  Manan
2  Adam
```

→ Only 'name' and 'Name' were printed. 'NAME' was not printed.

WHY IT IS IMPORTANT

Because of case sensitivity, Python treats each name differently.
This applies to:

- Variables
- Functions
- Classes
- Modules

COMMON MISTAKE

Using wrong case can cause errors.

✗ Wrong (raises an error):

```
Print("Hello")
```

```
NameError: name 'Print' is not defined
```

✓ Correct (no error):

```
print("Hello")
```

```
Hello
```

KEY TAKEAWAYS

1. Python is **case-sensitive**. `name` and `Name` are not the same.
2. Be **consistent** in using names in your code.
3. Even small case changes can lead to **errors**.
4. Always use correct case for built-in functions like `print()`.

© Copyright Intellipaat. All rights reserved.

Statements / Line Breaks

In Python, each **statement** is usually written on a separate line.
A new line marks the end of one statement and the beginning of another.

1 ONE STATEMENT PER LINE (RECOMMENDED)

This is the standard and most readable way in Python.

```
1  x = 10          # First statement
2  y = 20          # Second statement
3  sum = x + y     # Third statement
4  print(sum)      # Fourth statement
```

Each line is a separate statement.
Easy to read, debug and maintain.

2 MULTIPLE STATEMENTS IN ONE LINE (NOT RECOMMENDED)

Python allows multiple statements on one line using semicolon (;).

```
1  x = 10;  y = 20;  sum = x + y;  print(sum)
```

- It works, but reduces readability.
- **Avoid** this in real programs.

3 NO SEMICOLON NEEDED

Unlike C/Java, Python does **not require** a semicolon (;) at the end of a statement.

```
x = 10;
y = 20;
print(x + y);
```

✗

```
x = 10
y = 20
print(x + y)
```

✓

Both will work, but semicolons are not used in Python.

KEY POINTS

1. A new line indicates the end of a statement.
2. Python usually does not use semicolons.
3. Write one statement per line for clean and readable code.
4. Use semicolons only when you have a very specific reason.

EXAMPLE

Python's use of line breaks instead of semicolons makes code look clean and similar to natural language.

C / JAVA (uses semicolon)

```
int x = 10;
int y = 20;
int sum = x + y;
System.out.println(sum);
```

PYTHON (uses line breaks)

```
x = 10
y = 20
sum = x + y
print(sum)
```

Good habits make your Python code simple, readable and professional!

© Copyright Intellipaat. All rights reserved.

Introduction to Primitive Data Types

© Copyright Intellipaat. All rights reserved

Primitive Data Types

Primitive data types are the **basic building blocks** in Python.
They are used to store **simple values** like numbers, true/false values, and empty (no value) values.

REAL-LIFE ANALOGY

Just like we use different **containers** in real life to store different kinds of things, Python uses **different data types** to store different kinds of values.



Whole Numbers
(Integer)



Decimal Numbers
(Float)



True or False
(Boolean)



No Value / Empty
(None)

REAL-LIFE EXAMPLES

	Real-Life Value	Python Data Type	Why this type?
Age of a person	21	`int` (Integer)	Because age is a whole number (no decimal part).
Height of a person	5.8	`float` (Float)	Because height can have decimal values.
Is student present?	True	`bool` (Boolean)	Because the value is either **True** or **False**.
Middle name (not provided)	None	`None` (NoneType)	Because there is no value available.

These primitive data types are the **foundation** for all programs in Python.
Next, we will learn each of them in detail!

© Copyright Intellipaat. All rights reserved.

Integer (int)

Integer is a primitive data type used to store whole numbers (positive, negative or zero) without any decimal part.

KEY POINTS

- Stores whole numbers only.
- Can be positive, negative, or zero.
- No decimal point.

SYNTAX

```
variable_name = integer_value
```

Example: `age = 21`

EXAMPLES

Valid Integers

25 -10 0 999 -2500

All of the above are integers.

In Python

```
1  age = 21
2  temperature = -5
3  count = 0
4  students = 150
```

REAL-LIFE EXAMPLES



Age of a person
`age = 21`



Number of students in a class
`students = 40`



Temperature below zero
`temp = -8`



Number of wins in a game
`wins = 12`

QUICK TIP

If a number has a decimal point, it is not an integer.
For example, 5.0 is a float, not an integer.

```
x = 10      ✓ Integer
y = 10.0    ✗ Not Integer (It's float)
```

© Copyright Intellipaat. All rights reserved.

Float (float)

Float is a primitive data type used to store decimal numbers (numbers with a decimal point).

KEY POINTS

- Stores decimal numbers.
- Has a decimal point.
- Can be positive, negative, or zero.
- Used for more precise values.

SYNTAX

```
variable_name = float_value
```

Example: `height = 5.8`

EXAMPLES

Valid Floats

3.14 -2.5 0.0 99.99 -0.75

All of the above are floats.

In Python

```
price = 99.99
height = 5.8
temperature = -3.7
average = 76.45
discount = 0.25
```

REAL-LIFE EXAMPLES

Height of a person	`height = 5.8` feet
Price of a product	`price = 149.99` dollars
Body temperature	`temperature = 36.6` °C
Fuel in the tank	`fuel = 12.75` liters

QUICK TIP

- If a number has no decimal point, Python treats it as an integer.
- If it has a decimal point, it is treated as a float.

```
x = 10    → int
y = 10.0  → float
```

Use float when the value can have decimal places.

© Copyright Intellipaat. All rights reserved.

Boolean (bool)

Boolean is a primitive data type used to represent two values:

True or False.

KEY POINTS

- Stores one of two values: True or False.
- Used to represent conditions or logical states.
- Very important in decision-making and control flow.
- In Python, True and False are case-sensitive.

SYNTAX

```
variable_name = True | False
```

Examples:

```
is_active = True
is_admin = False
```

EXAMPLES

Valid Boolean Values

True
(Represents Yes, On, 1, or Positive)

False
(Represents No, Off, 0, or Negative)

In Python

```
1  is_logged_in = True
2  has_permission = False
3  is_student = True
4  is_raining = False
5  is_empty = False
```

In Python, True behaves like 1 and False behaves like 0 in calculations.

REAL-LIFE EXAMPLES

Situation	Boolean Value	Meaning
Is the door locked?	True	Yes, locked
Is it daytime?	True	Yes, it is daytime
Is it raining?	False	No, it is not raining
Is the answer correct?	True	Yes, the answer is correct
Is the device ON?	False	No, the device is off

QUICK TIP

Use boolean values in conditions, loops, and decisions.
Example: `if is_logged_in: ...`

```
if is_raining:
    print("Take an umbrella!")
```

Boolean values make your program smarter and more logical!

© Copyright Intellipaat. All rights reserved.

None (nonetype)

None is a special primitive data type that represents no value, empty value, or the absence of a value.

KEY POINTS

- Represents no value or empty value.
- Used when a variable has no assigned value yet.
- Returned by functions that do not return anything.
- It is not the same as 0, False, or an empty string.
- In Python, None is case-sensitive (only None).

SYNTAX

```
variable_name = None
```

Example:

```
result = None
```

EXAMPLES

In Python

```
1  name = None
2  age = None
3  result = None
4  data = None
5  response = None
```

What it looks like

Output

```
name      -> None
age       -> None
result    -> None
data      -> None
response  -> None
```

None means "nothing is stored here" or "value is not available yet".

REAL-LIFE EXAMPLES

Situation	Example in Python	Why None?
A form field not filled by the user	`middle_name = None`	The user didn't provide a value yet.
A package not assigned yet	`package = None`	No package has been assigned yet.
A missing database record	`record = None`	Record not found in the database.

QUICK TIP

Use None when you want to represent "no value" or "value is not available".

None vs Other Values

None (No value) ≠ 0 (Zero) ≠ False (Boolean False) ≠ "" (Empty String)

© Copyright Intellipaat. All rights reserved.

Dynamic Typing

© Copyright Intellipaat. All rights reserved

Dynamic Typing Mechanics in Python

Python is **dynamically typed**, which means you don't need to declare the data type.
Python **automatically decides the type** based on the value assigned to the variable.

KEY POINTS

- You don't declare data types in Python.
- The type is decided at **runtime** based on the value assigned.
- The same variable can hold values of **different types** at different times.
- Python changes the type **automatically** whenever a new value is assigned.

HOW IT WORKS?

Step	Code	Value Stored	Type	What Happens?
1	`x = 10`	10	`int`	x becomes an integer
2	`x = 3.14`	3.14	`float`	x now becomes a float
3	`x = "Hello"`	"Hello"	`str`	x now becomes a string
4	`x = True`	True	`bool`	x now becomes a boolean

The variable name (x) remains the **same**,
but its type **changes automatically**.

EXAMPLE IN PYTHON

```
1  x = 10
2  print(type(x))          # <class 'int'>
3
4  x = 3.14
5  print(type(x))          # <class 'float'>
6
6  x = "Hello"
7  print(type(x))          # <class 'str'>
```

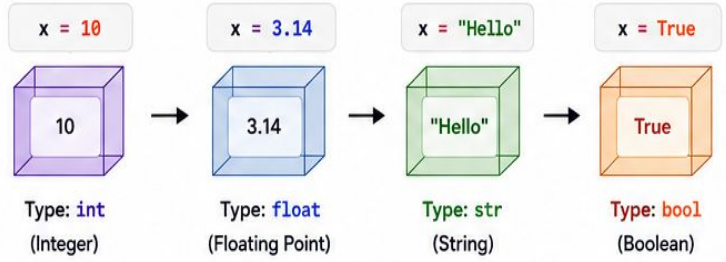
Output

```
<class 'int'>
<class 'float'>
<class 'str'>
```

© Copyright Intellipaat. All rights reserved.

Dynamic Typing Mechanics in Python

SAME VARIABLE, DIFFERENT TYPES



Each time a new value is assigned, Python updates both the value and the type.

COMPARED TO STATIC TYPING (e.g., C / Java)

Python (Dynamic Typing)	C / Java (Static Typing)
✓ No need to declare type.	✗ Type must be declared explicitly.
✓ Type decided at runtime.	✗ Type is fixed at compile time.
✓ Same variable can hold different types.	✗ Same variable cannot hold different types.

```
x = 10
x = 3.14
x = "Hello"  # All valid
```

```
int x = 10;
x = 3.14;       // Error
x = "Hello";    // Error
```

QUICK TIP

- Python is **flexible** and **beginner-friendly** because of dynamic typing.
- Just assign a value and start using the variable!

Remember: The variable name is just a **reference**. Python decides the **type**, not you!

© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved

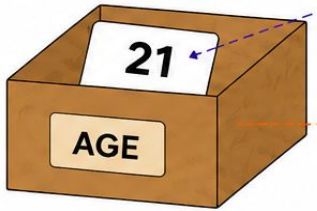
Variables

© Copyright Intellipaat. All rights reserved

What is a Variable?

A variable is a **named container** used to **store data values** in a program.
It helps us save data with a name so that we can use it later.

REAL-LIFE ANALOGY



Label on the box
= Variable Name
(Age)

Value inside the box
= Data Stored
(21)

WHY DO WE USE VARIABLES?

- To store data for later use.
- To give meaningful names to data.
- To make programs easier to read, write and manage.
- To update values when needed.

EXAMPLES IN PYTHON

```
age = 21              # storing an integer
name = "Manan"        # storing a string
height = 5.8          # storing a float
is_student = True     # storing a boolean
```

What's happening?

- We create variables (`age`, `name`, `height`, `is_student`).
- We assign values to them using `=`.
- Python stores these values and we can use them later.

VISUAL REPRESENTATION

Variable (Name)	Value (Stored)
age	21
name	"Manan"
height	5.8
is_student	True

In short: A **variable** is a **name** given to a value.
It helps us store, access and update data easily in our programs.



© Copyright Intellipaat. All rights reserved.

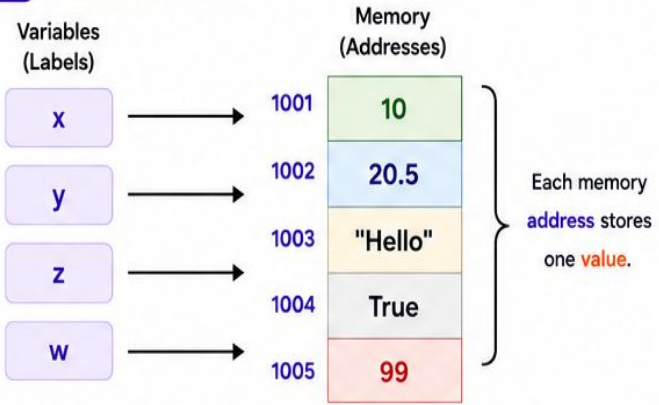
Variables as Memory References

In Python, variables do not store the actual value.
They act as references (labels) to a location in memory where the value is stored.

KEY POINTS

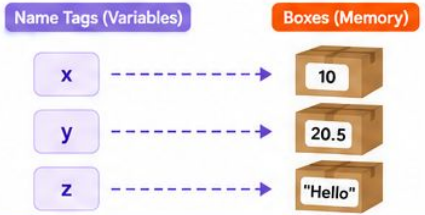
- A variable name is like a label.
- The label points to a memory location.
- The actual value is stored in that memory location.
- Multiple variables can refer to the same memory location.
- When the value changes, the reference may change.

HOW IT WORKS?



REAL-LIFE ANALOGY

Think of variables as name tags and memory locations as boxes.



If the value changes, the label may point to a different box.

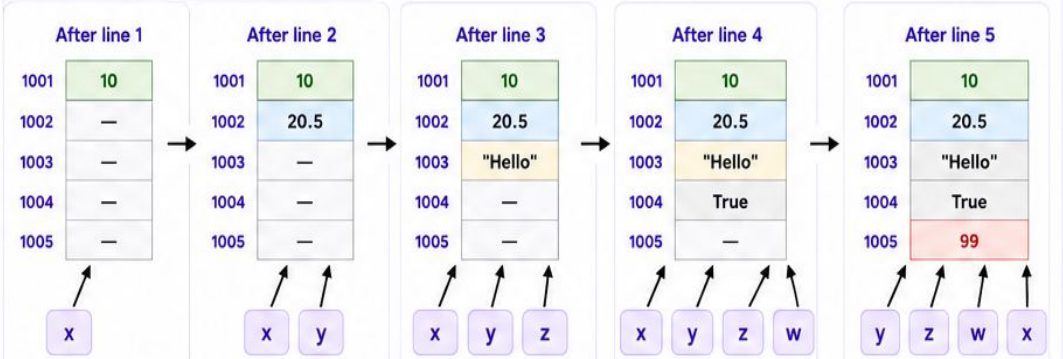
© Copyright Intellipaat. All rights reserved.

Variables as Memory References

EXAMPLE IN PYTHON

```
1  x = 10              # x points to address 1001
2  y = 20.5            # y points to address 1002
3  z = "Hello"         # z points to address 1003
4  w = True            # w points to address 1004
5  x = 99              # x now points to a NEW address 1005
6  print(x, y, z, w)   # Output: 99 20.5 Hello True
7
```

WHAT'S HAPPENING IN MEMORY?



QUICK TIP

- Choose meaningful names.
- Follow a consistent naming style (preferably snake_case).
- Keep names short but descriptive.

```
# Good
total_price = 2500
student_count = 42
# Bad
tp = 2500, sc = 42
```

REMEMBER

The variable name is just a **reference**. Python decides the **type** & stores the **value** in memory.

© Copyright Intellipaat. All rights reserved.

Variable Naming Conventions

Good variable names make your code easier to read, understand, and maintain.
Follow consistent **naming conventions** to write clean and professional code.

KEY GUIDELINES

- Use **meaningful** and **descriptive** names.
- Use letters, digits, and underscores (_).
- Names cannot start with a digit.
- Avoid using keywords and built-in function names.
- Use a consistent naming style throughout your code.
- Variable names are case-sensitive.

NAMING STYLES

Style	Example
snake_case	total_amount
camelCase	totalAmount
PascalCase	TotalAmount
UPPER_CASE	MAX_VALUE

In Python, **snake_case** is the most commonly used style.

© Copyright Intellipaat. All rights reserved.

Variable Naming Conventions

EXAMPLES IN CONTEXT

Purpose	Good Example	Why It's Good
Counting items	`item_count`	Clearly indicates it stores a count
User information	`user_email`	Descriptive and specific
Calculation result	`final_total`	Indicates it holds the final value
Boolean flag	`is_verified`	Starts with 'is_', easy to understand

THINGS TO AVOID

- Single letter names (except in small loops): `i, j, x, y`
- Ambiguous names: `data, value, info`
- Keyword names: `for, class, return`
- Built-in names: `list, str, sum, id`
- Inconsistent style: mixing `camelCase` and `snake_case`

QUICK TIP

Choose names that explain the **purpose** or **meaning** of the variable. Good names save time for you and others!

A good variable name is like a **good comment** it should tell the story of your code.

© Copyright Intellipaat. All rights reserved.

Handling User Interaction

© Copyright Intellipaat. All rights reserved

What is a User Interaction

User interaction means communication between the **user** and the **program**.
The user gives **input**, and the program produces **output (response)**.

THE BASIC IDEA



Every program you use in real life follows this simple cycle:

Input → Process → Output

REAL-LIFE ANALOGY



You enter your PIN (Input)
You give data to the ATM.

ATM processes your request (Program)
It checks and processes the data.

ATM shows your balance (Output / Response)
It gives the result to you.

EXAMPLES OF USER INTERACTION

Chat Application
You type a message (**input**) and see the reply (**output**).

Search Engine
You enter keywords (**input**) and get results (**output**).

Games
You press keys/buttons (**input**) and see action (**output**).

Online Shopping
You add items or details (**input**) and get order confirmation (**output**).

In short: **User interaction** makes programs useful and meaningful.
Without input and output, a program is of no use!

In the next slides, we will learn how Python handles input and output using `input()` and `print()`.

© Copyright Intellipaat. All rights reserved.

READING INPUT USING input()

The input() function is used to take input from the user during program execution.

HOW IT WORKS



1. Program asks for input
2. User types and presses Enter
3. Value is stored in the variable

Whatever the user types is taken as input and stored in the variable.

SYNTAX

```
variable = input("prompt for user: ")
```

- The text inside the quotes is shown to the user.
- The user types something and presses Enter.
- The typed value is returned by input() and stored in the variable.

IMPORTANT

By default, input() returns data as a string.

© Copyright Intellipaat. All rights reserved.

READING INPUT USING input() Examples

EXAMPLE 1: STRING INPUT

```
name = input("Enter your name: ")
```

```
# if the user enters any string,
# then name stores that string
```

Example Run

```
Enter your name: Manan
(Program continues..)
```

User input

Memory (after input)	
name	"Manan"

String value stored in name

EXAMPLE 2: NUMBER INPUT

```
age = int(input("Enter age: "))
```

```
# Here age stores the input
# in terms of an integer
# using int()
```

Example Run

```
Enter age: 21
(Program continues..)
```

User input

Memory (after input)	
age	21

Integer value stored in age

QUICK TIPS

- `input()` always returns a string.
- To work with numbers, convert using `int()`, `float()`, etc.
- The program waits for the user to type and press **Enter**.

NOTE

Pressing **Enter** without typing anything returns an empty string `""`.

© Copyright Intellipaat. All rights reserved.

DISPLAYING OUTPUT USING print()

The `print()` function is used to display **output** on the screen.

HOW IT WORKS



1. Program sends output to screen
2. Python processes it using `print()`
3. Output is displayed on the screen

`print()` can display text, values, variables, or a combination of both.

</> SYNTAX

```
print(value1, value2, ..., sep=' ', end='\n')
```

- `value1, value2, ...` : items to display (text, numbers, variables, etc.)
- `sep` (optional) : separator between items (default is a space)
- `end` (optional) : end character after output (default is new line)

IMPORTANT

By default, `print()` adds a **new line** after the output.

© Copyright Intellipaat. All rights reserved.

DISPLAYING OUTPUT USING print() Examples

EXAMPLE 1: SIMPLE OUTPUT

```
print("Hello World")
```

```
# prints the text Hello World
```

Output

```
Hello World
```

Explanation

The text inside quotes is displayed on the screen.

EXAMPLE 2: OUTPUT USING VARIABLES

```
name = "Adam"
print("Hello", name)
```

```
# name stores the string "Adam"
# Hello and the value of name
# are printed
```

Output

```
Hello Adam
```

Explanation

`print()` prints multiple items separated by a space.

In Short: Use `print()` to show results, messages, or values to the user. It is the primary way to produce output in Python programs.

Remember:

Programmer writes code → `print()` shows it to the user!

EXAMPLE 3: COMBINED INPUT AND OUTPUT

```
name = input("Enter your name: ")
print("Hello", name)
```

```
# takes input from user
# displays greeting
```

Sample Run

```
Enter your name: Adam
Hello Adam
```

Explanation

First, `input()` takes the name.
Then, `print()` displays the greeting.

© Copyright Intellipaat. All rights reserved.

Difference between Input() and print()

In Python, `input()` is used to take data from the user, while `print()` is used to display data to the user.

FEATURE	INPUT (input())	OUTPUT (print())
Purpose	Takes data from user	Displays data to user
Direction	User → Program	Program → User
Function Used	`input()`	`print()`
Return Value	Returns user input (as string)	Displays text, returns nothing
Example	`name = input("Enter name: ")`	`print("Hello")`

input() – Takes data from user



print() – Displays data to user



In short:

- Use `input()` when you want the user to give data to your program.
- Use `print()` when you want to show or display something to the user.

Note:

The `input()` function always returns the data as a **string**.

© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.